

InventiX — Supplier Side Handover

What is already done, and exactly what you need to change. 12 September 2026.

Already working: login, product catalog, and the whole customer side — stocks, suppliers, the restock popup, delivery tracking and POS report uploads. All tested against the real backend and database.

Your part: the supplier side — Listings, Orders, and the supplier Delivery queue.

1. Getting set up

1. Pull the latest `main` , then branch: `git checkout -b feat/supplier-integration`
2. Create `frontend/.env` from `frontend/.env.example` . Get the Supabase values privately from Pulindu. Set `EXPO_PUBLIC_API_URL` to **your own machine's** LAN address (`ipconfig`), e.g. `http://192.168.1.5:8000`
3. Create `backend/.env` the same way
4. Run both:

```
cd backend
.venv\Scripts\activate
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

cd frontend
npx expo start --clear
```
5. Log in as `demo.supplier@inventix.lk`

No SQL to run. We all share one Supabase project and it is already up to date. Never run `demo_data.sql` again — it would create duplicate orders.

2. The one change you definitely must make

Four screens run a write without catching a refusal. When the backend says no, the button spins for ever and the error never reaches the user. This happened on the customer side repeatedly and hid two real bugs for hours.

File	Buttons affected
<code>app/(supplier)/listings/add.tsx</code>	Add listing
<code>app/(supplier)/listings/[id].tsx</code>	Save, activate / deactivate
<code>app/(supplier)/orders/[id].tsx</code>	Confirm, reject
<code>app/settings/index.tsx</code>	Log out

Use the existing helper `src/hooks/useSubmit.ts` . The pattern, copied from `app/(customer)/stocks/add.tsx` :

```
const submit = useSubmit();

async function save() {
  const ok = await submit.run(() => createListing(input));
```

```

    if (ok) router.back();          // only navigate on success
  }

  // in the view
  <ErrorBanner message={submit.error} />
  <Button label="Save" loading={submit.busy} onPress={save} />

```

Delete the screen's own `busy` state when you do this.

3. What to test, feed by feed

Keep the backend terminal visible. Every screen action prints a line there: `200` , `201` , `204` are fine; `401` means a login problem, `403` wrong account type, `422` the app sent the wrong shape, `500` the backend crashed — read the red text.

Feed	Test
Listings	4 products appear (rice, milk powder, sugar, biscuits). Add a new one. Change a price. Switch one off and on again — it must deactivate, never delete, or past orders break.
Orders	The shop's order appears as new. Confirm it. Send another from the shop account and reject it — a reason of at least 5 characters is required.
Supplier delivery	Move a confirmed order through processing → put to delivery → on the way → delivered. Skipping a stage must be refused.
Both roles	With the shop on another phone: send an order and watch it appear on the supplier's screen without refreshing . Then advance a stage and watch the shop's progress bar move.

4. Traps we already hit

These cost hours on the customer side. If you see the same symptom, this is why.

Symptom	Cause
"Something went wrong on our side"	A 500. The real message is only in the backend terminal — always read it before guessing. Two of ours were SQL problems invisible to the tests.
Button spins for ever	Missing <code>useSubmit</code> (section 2).
Every screen: "Network request failed"	Backend not running, started without <code>--host 0.0.0.0</code> , or your LAN IP changed.
Screens empty, nothing in the backend terminal	Expo did not pick up <code>.env</code> . Restart with <code>npx expo start --clear</code> .
Data right, but only after pull-to-refresh	Realtime. Already switched on for orders, stock, notifications and listings — if you add a table, add it to the publication too.
Everything returns 403	Logged in as the wrong role for that screen.

5. Rules

- **Never edit a migration that has already run.** Database changes are a new numbered file in `database/migrations/`. Files in `functions/` and `policies/` are replaced in place.
- **Never commit a `.env` file**, and never put real UUIDs into `demo_data.sql`. Both are already ignored or placeholdered — check `git status` before committing.
- **Fix the cause, not the display.** If a number is wrong, it is wrong in that feed's `service.py`. The app must only show what the backend sends.
- **One commit per feed**, so any one can be undone on its own.
- Before saying something works: `cd backend && pytest -q` and `cd frontend && npx tsc --noEmit`.

6. Known gaps — not your bugs

These are unbuilt, already known, and being handled separately. Do not start them without asking.

- A rating is never saved. The prompt appears after the shop confirms receipt, collects stars, and throws them away — there is no endpoint. This is why a supplier's score will not move yet.
- No way to correct a saved POS product match once it is made.
- "My profile", "Change password" and "Help and support" open nothing.
- Push notifications are off. In-app notifications work; phone pop-ups need Firebase and a development build, which Expo Go cannot do.

If you use an AI assistant on this repo, point it at `CLAUDE.md` in the project root first. It carries the architecture rules, the setup commands and every trap above in the form the assistant needs, so it does not rediscover them one error at a time.

Fuller detail: [docs/11-integration-plan.pdf](#) section 5 lists every endpoint per feed. [docs/03-bug-map.md](#) maps a symptom to the file that owns it.